FH Salzburg
MultiMediaTechnology

Basics Game Development

Transformations

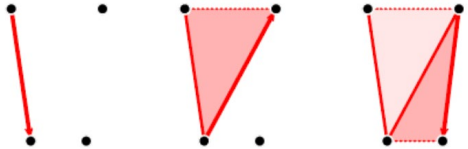
(Partly based on cg-slides from M. Tatzgern)

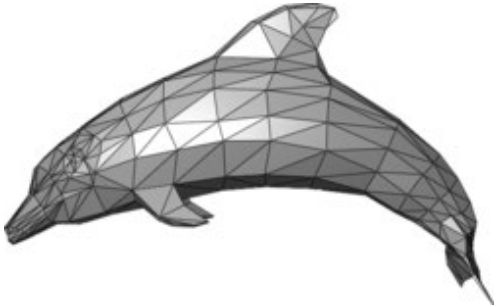
Technology
Health
Media

„Geometry“

- When we talk about "geometry" in GameDev, we basically mean a set of vertices (this holds for 2D to).
- In detail:
  - a **polygon mesh** is a **collection of <u>vertices</u>, edges and faces** that defines the shape of a polyhedral object. The **faces usually consist of triangles** (triangle mesh).
    - **Vertex**: a **position** (usually in 3D space) along with other information such as **color**, **normal vector** and **texture coordinates (UV-coordinates)**.
    - **Edge**: A connection between two vertices.
    - **Face**: A closed set of edges, in which a *triangle face* has three edges.

Triangle strip construction of a quad:





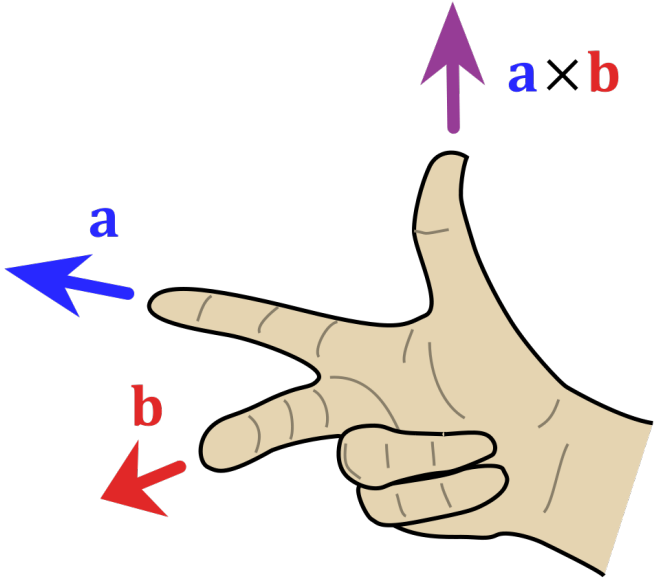
Transformations

Coordinates and Coordinate Systems

- In a 2D graphics system, you express **pixel coordinates** with two numbers: (X,Y). These are screen coordinates to indicate that each integer number X and Y corresponds to a row and column of pixels, respectively.
- If you want to describe a 2D coordinate system fully, you need a little more data, such as where (0,0) is on the screen, whether the X coordinate describes rows or columns, and in which direction the coordinates grow—to the left or right.
- **Since the display memory is organized in row order, it makes sense to locate the origin at the top left-hand side of the screen.**

Coordinates and Coordinate Systems

- You’ve probably heard of **3D coordinate systems** described as either left- or righthanded.
- Why is **handedness** important? When you move objects around your world, you’ll want to know where your positive Z-axis is and how it relates to the other two.



Matrix Mathematics

- A 3D world is filled with objects that move around. It would seem like an impossible task to set each vertex and surface normal of every polygon each time an object moves. **There's a shortcut**, it turns out, and it concerns matrices.
- Vertices and surface normals for objects in your world are stored in **object space**. As the object moves and rotates, the only thing that changes is the object's **transform matrix**. The **original vertices and normals remain exactly the same**.
- The object's transform matrix holds information about its position in the world and its rotation about the X-, Y- and Z-axis (in 2D only 1 axis).

Linear Transformations

A linear transformation is a function that changes vectors or matrices into new ones, using a formula with a specified format. The new vectors or matrices still behave the same way under addition and scalar multiplication. Translation is not a linear transformation because it changes how vectors or matrices behave under these operations. Translation is when you add a constant vector or matrix to every vector or matrix in a space.

- All linear transformations can be deffined as a matrix:

$$T(p) = \boldsymbol{A} \cdot p + t$$
$$T(\alpha \boldsymbol{u} + \beta \boldsymbol{v}) = \alpha T(\boldsymbol{u}) + \beta T(\boldsymbol{v})$$

- Translation **is not linear**!

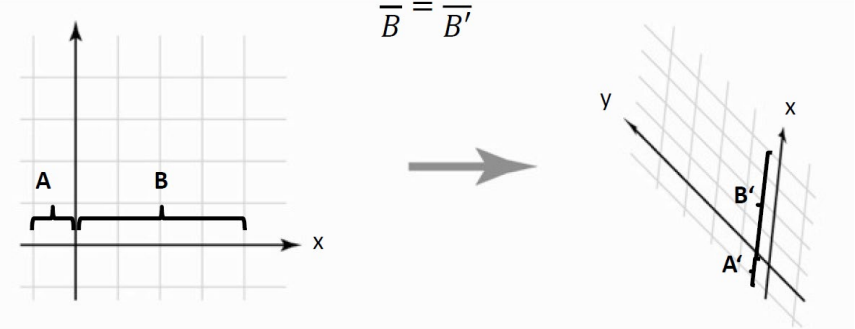
$$T(\alpha \boldsymbol{v}) = \alpha \boldsymbol{v} + \boldsymbol{u} \neq \alpha(\boldsymbol{v} + \boldsymbol{u}) = \alpha T(\boldsymbol{v})$$

Affine Transformations

- Transformations written as a combination of a linear transformation and a translation are called „affine“ transformations.

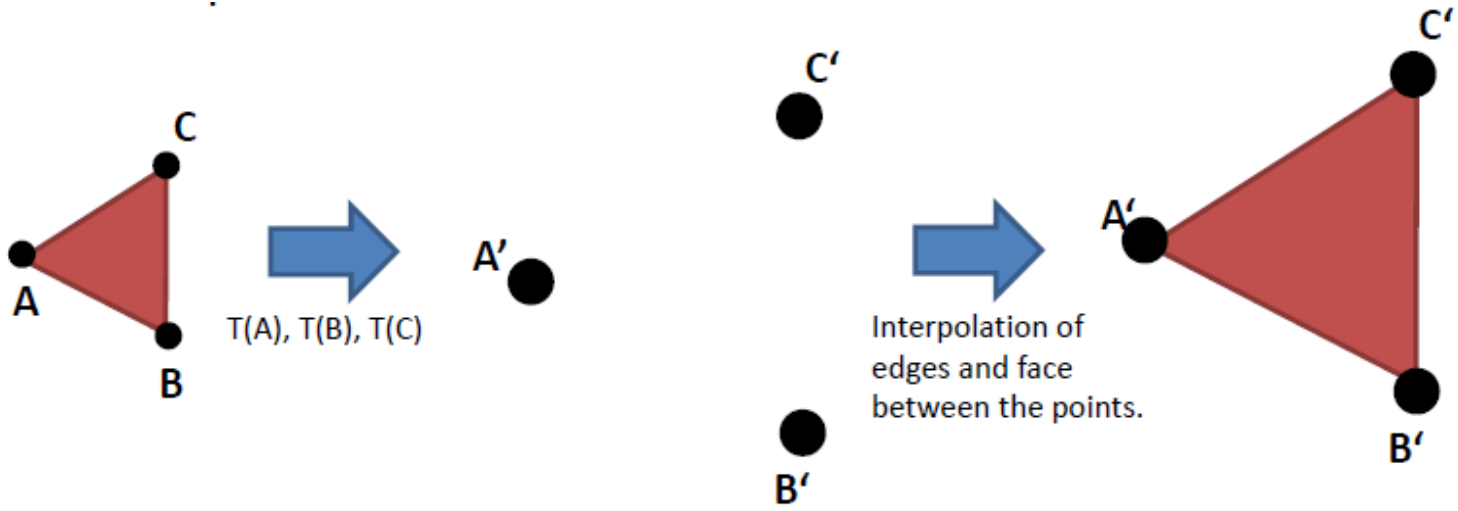
- Properties of affine transformations:
  - Lines stay lines (no curves)
  - Parallel lines stay parallel
  - Ratios of distances along lines are preserved

$$\frac{A}{B} = \frac{A'}{B'}$$



Importance of affine transformations

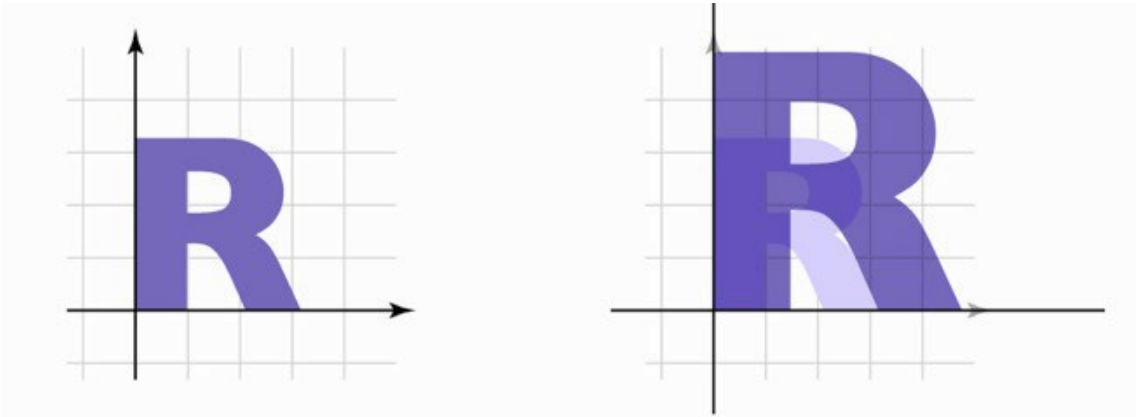
- Only the vertex-positions of a **geometry** need to be transformed.
- Edges and faces are just interpolations between the transformed points.



Uniform Scaling

- Uniform scale described by scale factor s:

$$S(s) \cdot v = \begin{pmatrix} s & 0 \\ 0 & s \end{pmatrix} \cdot \begin{pmatrix} v_x \\ v_y \end{pmatrix} = \begin{pmatrix} s \cdot v_x \\ s \cdot v_y \end{pmatrix}$$



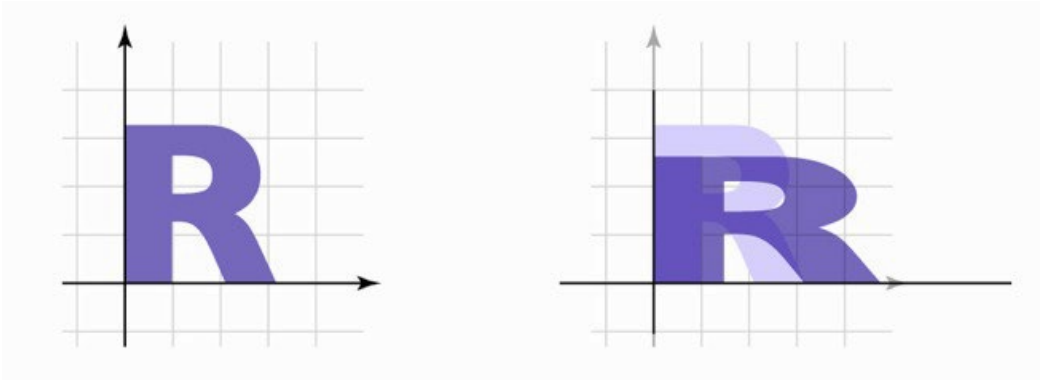
$$\begin{pmatrix} 1{,}5 & 0 \\ 0 & 1{,}5 \end{pmatrix}$$

Non-uniform Scaling

- Non-uniform scaling described via different scaling factors for each coordinate axis.

$$S(s) \cdot v = \begin{pmatrix} s_x & 0 \\ 0 & s_y \end{pmatrix} \cdot \begin{pmatrix} v_x \\ v_x \end{pmatrix} = \begin{pmatrix} s_x \cdot v_x \\ s_y \cdot v_y \end{pmatrix}$$

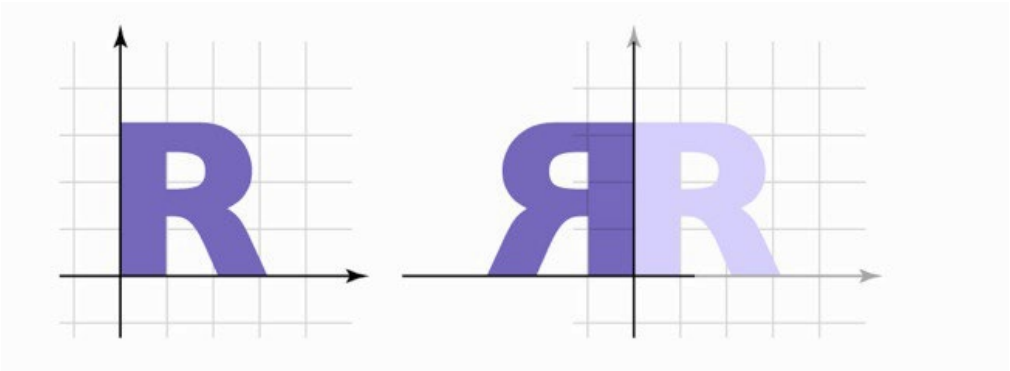
$$\begin{pmatrix} 1{,}5 & 0 \\ 0 & 0{,}8 \end{pmatrix}$$



Mirroring

- Mirroring can be regarded as a special case of non-uniform scaling with a negative scale factor to achieve a mirroring around an axis.

$$S\left((-1,1)^T\right) \cdot v = \begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} v_x \\ v_y \end{pmatrix} = \begin{pmatrix} -v_x \\ v_y \end{pmatrix}$$

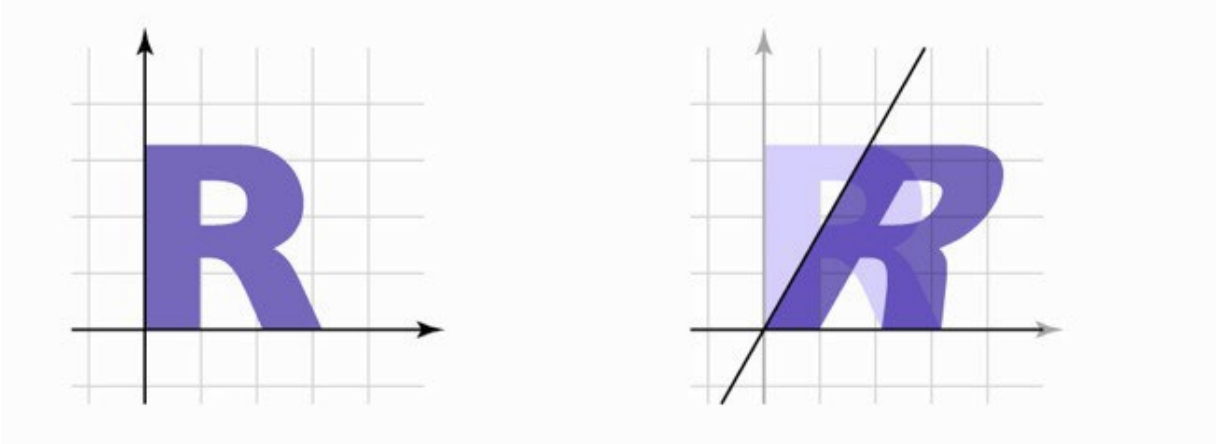


$$\begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix}$$

Shearing

- Shearing along an axis is described by a shear factor a

$$c_x(a) = \begin{pmatrix} 1 & a \\ 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} v_x \\ v_y \end{pmatrix} = \begin{pmatrix} v_x + a \cdot v_y \\ v_y \end{pmatrix}$$

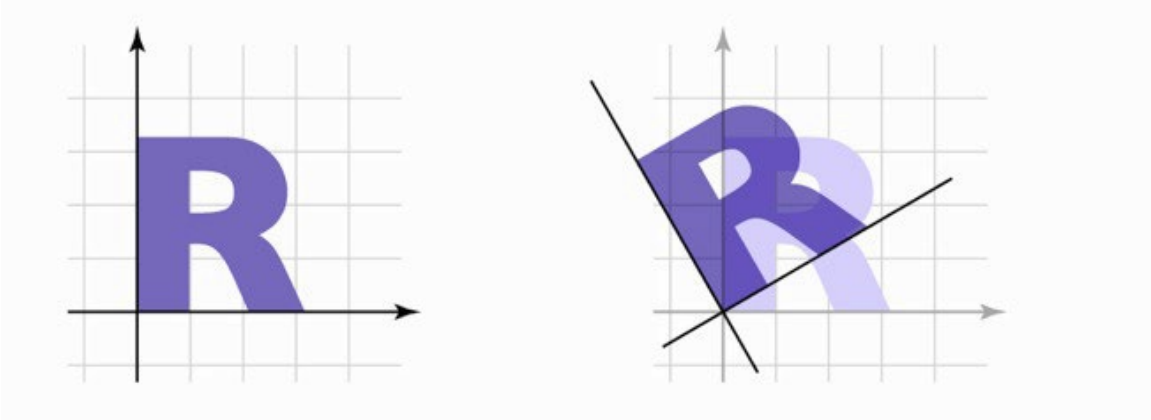


$$\begin{pmatrix} 1 & 0{,}5 \\ 0 & 1 \end{pmatrix}$$

Rotation

- Rotation described by rotation angle θ (counterclockwise)

$$R(\theta) \cdot v = \begin{pmatrix} cos\theta & -sin\theta \\ sin\theta & cos\theta \end{pmatrix} \cdot \begin{pmatrix} v_x \\ v_y \end{pmatrix} = \begin{pmatrix} v_x \cdot cos\theta - v_y \cdot sin\theta \\ v_x \cdot \sin\theta + v_y \cdot \cos\theta \end{pmatrix}$$



$$\begin{pmatrix} 0{,}8666 & -0{,}5 \\ 0{,}5 & 0{,}866 \end{pmatrix}$$

θ = 30 Grad

Chaining Transformations

- Transformation of 2D vector v_1

$$v_2 = Sv_1, \qquad v_3 = Rv_2$$

- Rewritten as

$$v_3 = R(Sv_1) \;\; or \;\; \boldsymbol{v_3} = (\boldsymbol{RS})\boldsymbol{v_1}$$

- Matrix multiplication is associative, i.e., we can calculate transformation matrices without transforming any points.

Important concept for computer graphics, i.e., compute chained transformation once for all points transformed with the same matrices.

Transformation Order (Rule of Thumb)

Always translate last. If you want to place an object in a 3D world, you always perform your **scale first , rotations second, and translations third**. Use “**SRT**” as a helpful mnemonic, standing for scale, rotate, translate.

Inverse Transformation

- Transformations can be reversed by inverting the transformation matrix.

$$v_3 = RSv_1$$

$$v_1 = (RS)^{-1} \; or \; v_1 = S^{-1}R^{-1} \; v_3$$

- Inverse transformations are applied in inverted order.

Chaining of Affine Transformations

- Problem:
  - Translation is required
    But translation is non-linear and cannot be described with a matrix. Or maybe we can?
- Idea:
  - Add an additional dimension w
    Place our known euclidean space at w=1
    → The new coordinates are called homogenous coordinates

$$\begin{pmatrix} v_x \\ v_y \end{pmatrix} \rightarrow \begin{pmatrix} v_x \\ v_y \\ 1 \end{pmatrix}$$

Homogenous Coordinates

- Our linear transformations can be described by a matrix by adding a column and row. Output does not change from non-homogenous case.

$$\begin{pmatrix} a & b & 0 \\ c & d & 0 \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} v_x \\ v_y \\ 1 \end{pmatrix} = \begin{pmatrix} av_x + \ bv_y \\ cv_y + dv_y \\ 1 \end{pmatrix}$$

Homogenous Coordinates

- Now the additional column can be used to describe a translation:

$$\begin{pmatrix} 1 & 0 & t \\ 0 & 1 & s \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} v_x \\ v_y \\ 1 \end{pmatrix} = \begin{pmatrix} v_x + t \\ v_y + s \\ 1 \end{pmatrix}$$

→ Effect of a translation of 2D points is described by a linear transformation in 3D

Homogenous Coordinates in 3D

- 3D Translation: $\begin{pmatrix} 1 & 0 & 0 & Translation.x \\ 0 & 1 & 0 & Translation.y \\ 0 & 0 & 1 & Translation.z \\ 0 & 0 & 0 & 1 \end{pmatrix}$

Position Vector and Direction Vector

- Until now we did not differentiate between position vectors and direction vectors when using transformations.

$$T(p) = A \cdot p + t$$

- Difference between position vector and direction vector is that a position is influenced by a translation, while a direction is not.

$$\begin{pmatrix} M & t \\ O^T & 1 \end{pmatrix} \begin{pmatrix} p \\ 1 \end{pmatrix} = \begin{pmatrix} Mp + t \\ 1 \end{pmatrix}$$

- With our new affine transformation with homogenous coordinates, we can use w to differentiate between point (w=1) and vector (w=0).

$$\begin{pmatrix} M & t \\ O^T & 1 \end{pmatrix} \begin{pmatrix} v \\ 0 \end{pmatrix} = \begin{pmatrix} Mv \\ 0 \end{pmatrix}$$

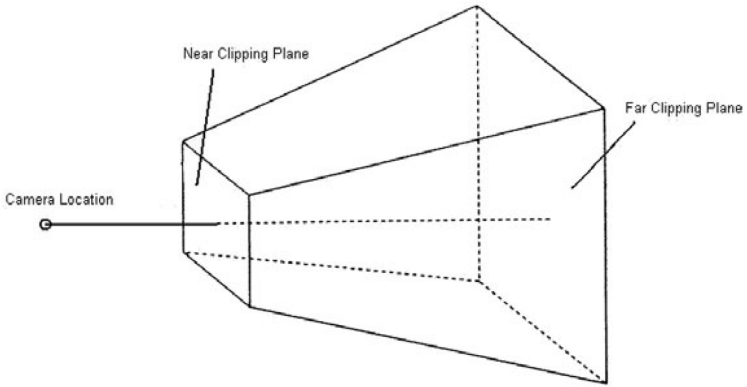
Attention: You can use a Vector2 (or Vector3) during your usual workflow in modern game-engines until you enter the „GPU-World“.

2D vs. 3D Graphics

- Similar concepts: vectors and matrices
- MVP Matrices:
  - **M**odel Matrix (Transform Matrix) („object to world“)
  - **V**iew Matrix (Camera Matrix) („world to view“)
  - **P**rojection Matrix („view to screen“)
- This matrices are usually applied to the geometry through **small programms** that are running directly **on the graphics-card (GPU)**. We call this programms **„Shader“.**
- We will learn more about different shader-types and the graphics-pipeline in the upcomming lecture.

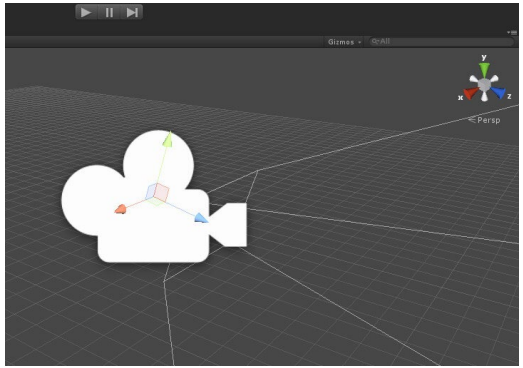
What is a Frustum?

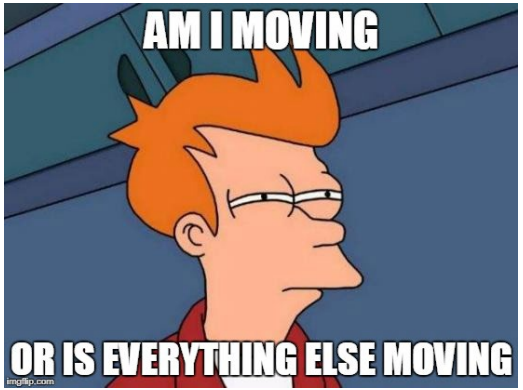
- Imagine sitting in front of a computer screen and seeing four lines coming from your eyeball and intersecting with the corners of the screen. These lines continue into the 3D world. You have a pyramid shape with the point at your eyeball and its base somewhere out in infinity. Clip the pointy end of the pyramid with the plane of your computer screen and form a base of your pyramid at some arbitrary place in the distance. This odd clipped pyramid shape is called the **viewing frustum**.



View Transform

- If you are going to render a scene, you need to have a **camera**. That camera must have an orientation and a position just like any other object in the world.
- Similar to any other object, the camera needs a transform matrix that converts world space vertices to camera space.
- **The camera is a lie!** Remember that the camera's view transform is a matrix, just like any other. We are applying the view-matrix to all objects in our scene. This basically means, we are moving all objects in the game-world but not the camera.

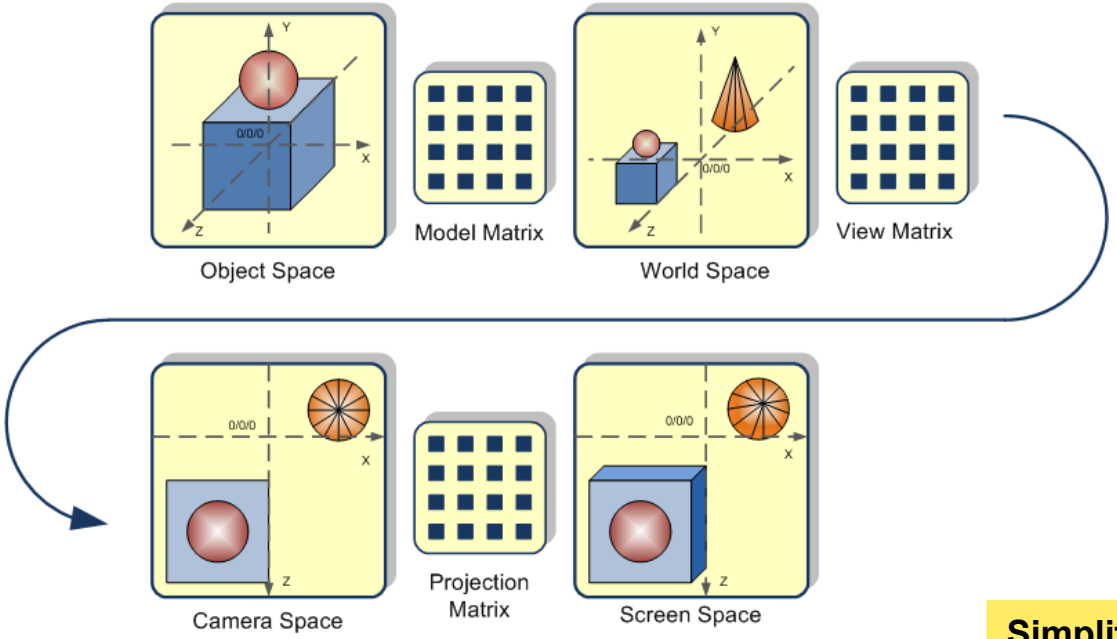




Projection Transform

- Using the world transform and the view transform, vertices from object space can be transformed into vertices from world space and then transformed into camera space.
- Now we need to take all those 3D vertices sitting in camera space and figure out **where they belong on your computer screen** and which objects sit in front of other objects. The view frustum will help determine what gets drawn and what gets culled.
- Important variables for selecting the the “type” and “lense” of the camera:
  - **Field of View:** Expressed in radians, this is the width of the view angle.
  - **Aspect Ratio:** This is the aspect ratio of your screen.
  - **Near/far clipping plane:** This is the distance between your eye and the near/far view plane (see “What is a frustum” Slide).

Model → View → Projection - Summary



Simplified model!

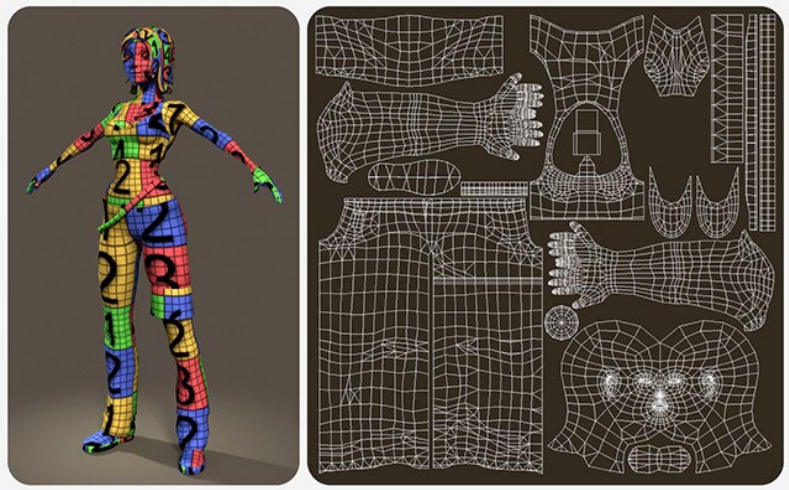
Quartenion Mathematics

- Euler Angles have a critical weakness when it comes to smooth interpolation between two orientations.
- Orientation can be expressed as three angles: yaw, pitch, and roll.
- There is a special mathematical construct known as a quaternion, and almost every 3D engine supports its use. A quaternion is a fourth-dimensional vector, and it can be visualized as a rotation about an arbitrary axis.
- Quaternions must be converted to transformation-matrices before they hit the graphics card.

Textures

Textures

- A texture is a piece of two-dimensional art that is applied to geometry. Each vertex gets a texture coordinate. Texture coordinates are conventionally defined as **(U,V) coordinates**, where U is the horizontal component and V is the vertical component.
- UV-Coordinates are in [0.0,1.0] where (0.0,0.0) signifies the top left corner and grows to the right and down.
- Each Vertex gets a Texture-Coordinate for every Texture.
- Textures can also serve as a data-source for normal maps, bump maps, clipping maps, noise, static light, …

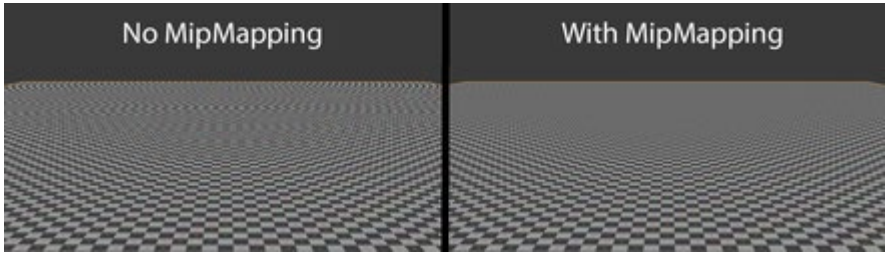


Textures

- A Texture can be interpreted as an array of color values.
- Types of Textures:
  - **1D-Textures**: Mostly used for color lookups. Only the U coordinate is used. It‘s a 1*n sized texture.
  - **2D-Textures**: The „common“ texture format with UV coordinates.
  - **3D-Textures**: A 3-dimmensional cube with an additional W coordinate. A 3D-Texture is normally built from a stack of interpolated 2D-images.

Mipmaps

- In computer graphics, mipmaps are pre-calculated, optimized sequences of images, each of which is a progressively lower resolution representation of the previous.
- The height and width of each image, or level, in the mipmap is a factor of two smaller than the previous level.
- Image showing how mipmaps reduce aliasing at large distances. (Aliasing causes a Moiré pattern in the left image.)



Graphic File Formats

- **PNG files are the format of choice for most 2D games today**, but it is worth taking a brief look at other common file formats (you know them from “multimedia” anyways).
- **Bitmap (BMP)** files are the most basic of the image file formats. For all practical purposes, they simply store the raw image data as a 2D array of colors. No transparency support and big file sizes.
- **TGA/TARGA** Developed as a native format for early graphics cards, Truevision graphics adapter (TGA) and Truevision advanced raster graphics adapter (TARGA) files allow for both raw and lossless compression. Simple in structure, TGA files were historically used for textures in 3D games.
- Unlike GIF and PNG, the Joint Photographic Experts Group (**JPEG or JPG)** image format utilizes lossy compression. That is, as the image is compressed, the original detail is lost and cannot be recovered.

Sprites

- A sprite is simply an image that we can use in our game.
- Sprite Generation for Games:
  - **Raster Based**: The bitmapped images are created by an artist, pixel by pixel, in a graphics editor such as Adobe Photoshop.
  - **Vector-Based**: Currently, vector-based sprite generation is more common with modern 2D games. The artist uses a graphics package such as Adobe Illustrator or Adobe Character Animator to draw vector graphics.
  - **3D-Based**: A third possibility for sprite generation is to create the image in 3D first, then take a snapshot of the rendered 3D image and save it as a image.

Sprite Sheets

- For efficiency, multiple sprites (or animation frames) are often grouped together in a single image file called a sprite sheet.
- When it comes to drawing the sprite to the screen, we need to consider the three representations of the data:
  1. **image file:** the compressed data as it exists in the file system (usually as a PNG image file);
  2. **source data:** the bitmap loaded into memory along with any information needed to track the bitmap (Texture-object in SFML).
  3. **destination data:** the information about where and how the individual sprite is drawn to the screen (Sprite-object in SFML).

FH Salzburg
MultiMediaTechnology

Basics Game Development

Transformations - End

Technology
Health
Media

© 2025 Fachhochschule Salzburg GmbH
www.fh-salzburg.ac.at